

BUILDING REST APIs WITH DJANGO

FIVE-DAY LEARNING AGENDA · VERSION 2 (REBALANCED)

TASK 02 · V2

EXPLAIN.
PRACTISE.
BUILD.

ORGANIZATION	TRACK	PROGRAM DATES	VERSION
Tuwaik Club at KFUPM	Modern Software & Cloud	October 11-15, 2026	2.0 · October 6, 2026 (replaces v1 of Sep 30)

01 PROGRAM OUTCOME (UNCHANGED)

BY THE END OF DAY 5

Participants can design, build, test, document, and deploy a database-backed REST API using Django REST Framework, including token authentication, validated CRUD operations, and per-user permissions.

02 FIVE-DAY LEARNING JOURNEY

DAY	FOCUS	PROJECT INCREMENT	DAILY PROOF
1 · Oct 11	Foundations and setup	Repository, Django/DRF project, Task model, migrations, admin data	Server runs; Task records persist
2 · Oct 12	Core API: CRUD + validation	Serializer, routes, complete CRUD endpoints, validation rules	CRUD checklist passes; invalid input returns 400
3 · Oct 13	Security	Token auth, ownership, per-user data isolation	Two-user security check passes
4 · Oct 14	Testing and configuration	8 automated tests, settings from the environment, deployment packages	8 required tests pass
5 · Oct 15	Publish and demonstrate	Production deployment, smoke test, final README, demonstration	Live URL and final evidence pass

03 DELIVERY MODEL

DAILY SESSION (ABOUT 3 HOURS)

BLOCK	DAYS 1-4	DAY 5
Hour 1 · ~60 min	Explanation + live demo	Explanation + live deployment of the reference project
Hour 2 · ~60 min	Exercise (Library API): core tasks first	Guided deployment of every participant's project
Hour 3 · ~60 min	Project milestone (Task API)	Verify, README, submission, live demo

Short breaks between blocks. Each day keeps ~10-20 min of buffer for beginners.

FACILITATION RULES

- Short explanations immediately followed by visible implementation.
- One project repository grows across all five days.
- Exercise tasks are marked **core** or **stretch**; the project milestone is what is assessed.
- Each day ends with a practical exit check, not a knowledge quiz.
- A known-good checkpoint lets any participant start the next day on time.
- No optional extensions before the day's required gate passes.

04 WHAT CHANGED FROM VERSION 1

Version 1 planned 45-60 min of explanation and 2-2.25 h of practice. Checked against a strict **1 h explanation + 1 h exercise + 1 h project** day for first-time API learners, Days 3-5 did not fit. Content was moved, not removed: the final outcome and the Definition of Done are unchanged.

DAY	VERSION 1	VERSION 2	WHY
2	CRUD only	CRUD + validation (error messages, due date not in the past)	Validation is serializer work; Day 2 had spare time
3	Security + validation ; 8 project TODOs, 11-step check	Security only; 4 core TODOs (+2 stretch), 8-step check	Explanation and project were ~75 min each
4	Tests + README + env config + partner review + deploy rehearsal	Tests + env config + deployment packages; README → Day 5; review and rehearsal → optional homework	Project alone was ~2 h
5	Project deploy + demos after an optional exercise	Hour 2 is a guided deployment ; hour 3 verify, README, submission, demos at 2-3 parallel stations	Deployment + demos needed both practice hours
1-5	All exercise tasks equal weight	Tasks labelled core / stretch; Day 1 ORM practice, Day 3 IsOwner + owner-as-username are stretch; Day 5 static-file/CSRF settings pre-written	Buffer of ~10-20 min per day for beginners

PERSONAL TASK MANAGEMENT REST API (UNCHANGED SCOPE)

Each authenticated user manages only their own tasks. The final API includes a persistent Task model, complete CRUD operations, token authentication, per-user isolation, input validation, at least eight automated tests, a reproducible GitHub repository, a clear README, and a public HTTPS deployment. A frontend is not required; optional features never block completion.

DAY 1

FOUNDATIONS, SETUP, AND PROJECT START

Sunday, Oct 11

DAILY LEARNING OUTCOME

By the end of the session, participants can explain the basic HTTP request-response flow, run a reproducible Django REST Framework environment, and persist Task records through a correctly migrated data model.

HOUR 1 · EXPLANATION + LIVE DEMO (~60 MIN)

10 min	The finished project first: request → Django → DRF → ORM → database → JSON + status code
10 min	HTTP verbs on the Task resource: GET reads, POST creates, PUT/PATCH change, DELETE removes
10 min	Django structure: project (config) vs app (tasks); DRF on top
25 min	Live demo: model → makemigrations → migrate → admin → a few ORM queries
5 min	Reproducibility: virtual environment, requirements.txt, .gitignore, secrets outside Git

HOUR 2 · EXERCISE (~60 MIN)

Core

- Complete the Book model (4 fields)
- makemigrations, migrate, load sample data (9 objects)
- Register Book in the admin (columns, filter, search)

Stretch / optional

- ORM practice file: 5 query functions with a self-check

HOUR 3 · PROJECT MILESTONE (~60 MIN)

Core

- Copy the starter into the participant's own folder; venv; install
- P1-P5: register apps, status choices, Task fields, __str__, admin
- Migrations, two users, sample tasks; first push to GitHub (may finish in the first 10 min of Day 2)

PROJECT FEATURE ADDED

Repository and reproducible environment; Django/DRF project; tasks app; Task model (id, title, description, status, due_date, owner, created_at, updated_at); migrations; admin; sample data.

COMPLETION CRITERIA (EXIT CHECK)

- Fresh terminal activates the environment and starts the server
- migrate reports no pending migrations
- Task fields and status choices match the approved scope
- Admin creates, updates, and persists Task records with owners
- Repository is pushed with no secrets or environment folder

NEXT-DAY CONNECTION

Day 2 turns this model into an API resource: serializer, routed CRUD endpoints, and validation.

DAY 2

CORE API: CRUD AND VALIDATION

Monday, Oct 12

DAILY LEARNING OUTCOME

By the end of the session, participants can convert Task objects to validated JSON, implement routed create, list, retrieve, update, partial-update, and delete operations, and reject invalid input with clear field-level errors.

HOURL 1 · EXPLANATION + LIVE DEMO (~60 MIN)

10 min	The serializer as translator (model <-> JSON) and gatekeeper
15 min	ViewSets and routers; the CRUD contract with its status codes
10 min	One POST traced end to end; PUT vs PATCH
15 min	Validation: built-in rules, extra_kwargs messages, validate_<field>, self.instance (create vs update)
10 min	Live demo in the browsable API: every endpoint, a 400 with field errors, a 404

HOURL 2 · EXERCISE (~60 MIN)

Core

- BookSerializer with a read-only related field
- BookViewSet + router: six operations
- Two validation rules: clear blank-title message; published year not in the future

Stretch / optional

- Ordering A-Z with a migration
- 9-request CRUD checklist

HOURL 3 · PROJECT MILESTONE (~60 MIN)

Core

- Morning: copy the Day 2 checkpoint
- P1-P6: ordering, TaskSerializer, TaskViewSet, perform_create, router, URLs
- P7-P8: title error messages; due date not earlier than today on create
- 10-request checklist in the browsable API (session login as a development bridge)

PROJECT FEATURE ADDED

Task serializer, router, and endpoints for GET/POST /api/tasks/ and GET/PUT/PATCH/DELETE /api/tasks/{id}/; validation for nonblank title, allowed status, and creation due date.

COMPLETION CRITERIA (EXIT CHECK)

- Routes resolve without manual URL duplication
- Create returns 201; retrieve 200; unknown id 404
- PUT/PATCH change the intended fields; DELETE returns 204
- Owner is read-only and assigned by the server
- Blank title, invalid status, and past creation due date return 400 with field errors

NEXT-DAY CONNECTION

Today every logged-in user sees all tasks. Day 3 adds token authentication and per-user isolation.

DAY 3

AUTHENTICATION, OWNERSHIP, AND PERMISSIONS

Tuesday, Oct 13

DAILY LEARNING OUTCOME

By the end of the session, participants can secure the API with tokens, isolate each user's data, and enforce ownership at query and object level.

HOURL 1 · EXPLANATION + LIVE DEMO (~60 MIN)

10 min	Authentication vs authorization; 401 vs 403 vs 404 in this program
10 min	DRF tokens: login endpoint, Authorization: Token <key> header
10 min	Postman: environment with base_url and token
20 min	Attack-and-fix demo with two users: filter get_queryset by request.user; owner in perform_create
10 min	Defense in depth: read-only owner, object permissions

HOURL 2 · EXERCISE (~60 MIN)

Core

- Token authentication: app, settings, login endpoint
- Ownership: get_queryset + perform_create

Stretch / optional

- User-aware validation (target date, no duplicate book per user)

HOURL 3 · PROJECT MILESTONE (~60 MIN)

Core

- Morning: copy the Day 3 checkpoint
- P1-P4: authtoken, authentication classes, token endpoint, get_queryset
- 8-step two-user security check in Postman

Stretch / optional

- P5: IsOwner object permission
- P6: owner shown as username

PROJECT FEATURE ADDED

Token login endpoint; protected task routes; automatic owner assignment; per-user list and detail isolation; cross-user update/delete protection.

COMPLETION CRITERIA (EXIT CHECK)

- Valid credentials return a token; no token returns 401
- A client-supplied owner is ignored
- Each user's list contains only their own records
- Cross-user retrieve/update/delete is blocked (404) without leaking data

NEXT-DAY CONNECTION

Day 4 converts today's manual security checks into repeatable automated tests.

DAY 4

TESTING AND CONFIGURATION

Wednesday, Oct 14

DAILY LEARNING OUTCOME

By the end of the session, participants can verify API behaviour with automated tests, read a failing test to find a bug, and move configuration and secrets into environment variables.

HOURL 1 · EXPLANATION + LIVE DEMO (~60 MIN)

10 min	A test as an executable promise: arrange, act, assert response and database
10 min	APITestCase, setUp, test isolation, tokens in tests; positive and failure tests
20 min	Live demo: a 401 test, a cross-user test; break the app and read the failure
15 min	Configuration from environment variables; .env.example; deployment packages
5 min	Preview of Day 5 and the optional homework

HOURL 2 · EXERCISE (~60 MIN)

Core

- 4 required tests for the reading-list API (tests only)

Stretch / optional

- 4 more tests
- "Break it on purpose": watch the tests catch a security bug

HOURL 3 · PROJECT MILESTONE (~60 MIN)

Core

- Morning: copy the Day 4 checkpoint
- P2: 7 required tests (2 built with the instructor); 8 with the example
- P3-P5: settings from environment, .env.example, deployment packages + pip check

Stretch / optional

- 7 more tests

PROJECT FEATURE ADDED

At least eight automated API tests; settings from environment variables; .env.example; deployment packages in requirements.txt. The README template arrives today and is completed on Day 5.

COMPLETION CRITERIA (EXIT CHECK)

- Tests cover unauthenticated access, create/owner, own-only list, cross-user access, update, delete, validation
- At least 8 meaningful tests pass; only stretch tests may be skipped
- Settings read secrets from the environment; .env.example has placeholders only
- pip check reports no broken requirements

NEXT-DAY CONNECTION

Optional homework (~15 min): open the Neon and Render accounts and read the deployment guide.

DAY 5

DEPLOY, VERIFY, AND DEMONSTRATE

Thursday, Oct 15

DAILY LEARNING OUTCOME

By the end of the session, participants can release the API to a public HTTPS environment, verify production behaviour and persistence, correct deployment issues, and demonstrate the completed system with evidence.

HOUR 1 · EXPLANATION + LIVE DEMO (~60 MIN)

10 min	Deployment = the same app under production configuration
25 min	Live demo: deploy the reference project on Render + Neon
10 min	Reading logs: configuration, dependencies, startup, hosts, database
10 min	Smoke tests and persistence; the README as part of the product
5 min	The 5-minute live demo: claim → action → proof

HOUR 2 · GUIDED DEPLOYMENT (~60 MIN)

Core

- Guided deployment (replaces the exercise): copy 3 files; P1 DATABASE_URL; P2 .env.example
- Neon database, Render web service, environment variables, production users

Stretch / optional

- Library exercise for Day 5 as homework

HOUR 3 · VERIFY, README, DEMO (~60 MIN)

Core

- Smoke test against the live URL (10/10)
- Persistence check after a restart
- P3: complete README with live URL and deployment section
- Submission + live demo at one of 2-3 parallel review stations

Stretch / optional

- One small quality improvement after every gate passes

PROJECT FEATURE ADDED

Public HTTPS API with a persistent PostgreSQL database; production configuration (WhiteNoise, static files and CSRF settings pre-written); smoke-test record; final README; demonstration.

COMPLETION CRITERIA (EXIT CHECK)

- Public HTTPS URL reachable; production migrations applied
- Token auth and all CRUD operations work in production
- Isolation and validation checks pass live; a record survives restart/redeploy
- README lets a new reviewer set the project up from a fresh clone
- Submission complete; demo runs without instructor code changes

PROGRAM COMPLETION

Program completion: a portfolio-ready backend and a repeatable workflow for future REST APIs.